

To make it boot properly, execute the following command:

```
./linux ubd0=debian_root_fs
```

When the boot completes, you will be asked for the login details — the user name is ‘root’ with no password.

You can type all the commands that you use to execute on a normal Linux OS. Even if you crash this, it will only crash the UML instance; so you can re-execute the Linux executable.

Multiple UML instances

You can boot a single UML instance using the file system provided. But if you try to start a second UML instance with the same file system, booting fails, showing a ‘file system error’. To start multiple UML instances, you can provide a copy of this root file system to each UML instance, which will comprise huge files. So let’s use a concept called COW (copy on write) files as a workaround. Let’s modify the command line parameters of UML to enable them to use COW files.

Start the first instance as follows:

```
./linux ubd0=cow1,debian_root_fs
```

Start the second instance as follows:

```
./linux ubd0=cow2,debian_root_fs
```

After executing, you will see two files getting created — cow1 and cow2. These are COW files used by each UML instance to make use of the existing root file system. Any change made by the first UML instance to the root file system will not be visible to the other UML instance. This enables you to save disk space and memory, because each UML instance will see both its individual COW files and root file system, so it will get a mixed view of both files. Any changes made by a UML instance will go to its respective COW file. So when the root file system is seen, what will be seen is a mixed view of both the root file system and COW files. So suppose UML1 has installed app1 on the file system, then that app will go to its COW file and will not go to the root file system. Hence, UML2 will not see that app or any changes from UML1—this is true for all UML instances.

This has one big advantage—any corruption that a UML instance makes will happen to the COW files and not to the actual file system, so this COW file can be discarded. If you see the size of the COW files, they look large but are actually sparse files. They actually occupy the disk space of only the amount of data that is modified by the UML instance. So having multiple COW files will actually take up less space than having individual copies of the file system for each UML instance.

Kernel debugging in UML

UML is a Linux kernel in itself, but debugging it is a

simple process for the host on which it is executing, just like any other process. You can debug UML using GDB, as if it is any other process. With this feature, you can learn many different parts of the Linux kernel such as process management, memory management, scheduler, file system, etc. We will see a simple example of how to debug UML. First, start an instance of UML in the terminal. From another terminal, note down the PID of the UML instance using the `ps -aux` command. Then start GDB (you need root permission to debug UML modules) with the Linux executable as its parameter.

Let us assume that you want to debug the `ls` command and see what is happening inside. To do that, you need to put a breakpoint to the `sys_clone` function in the Linux kernel. This is a common entry point for all commands executed on the command line. First, list the source code of `sys_clone` so that GDB reaches the file in which this function is present. Type the following command to list the source code of `sys_clone`:

```
list sys_clone
```

Press `Enter` continuously to find the line in which the `do_fork` function is called and note down the line number. Then type the following command to put the breakpoint at that line:

```
break <line_number>
```

Now, we need to attach GDB to a running UML instance. To do this, type the following command with the PID of the UML process, which was noted previously:

```
attach <UML_PID>
```

This will stop the running UML process. To start it again, type ‘continue’ on the GDB terminal.

After putting the breakpoint at that line, go to a UML terminal and type the `ls -l` command. Your UML instance will stop and the breakpoint will get hit in the GDB terminal. You can now use GDB commands to see the value of variables in the kernel code and learn about all the features of the kernel.

You can even write kernel modules, put breakpoints in those modules, and can debug them before they are actually inserted in a physical machine running the Linux kernel. You can even debug kernel drivers and test them virtually in UML.

Networking in UML

UML provides several interesting networking features in its virtual environment. If you were to study or build any network using physical machines, then you would need at least two physical machines with network interfaces on them. The virtual environment in UML provides a way to dynamically add or remove multiple network interfaces in it. Let us look at some way of using the networking environment in UML.